Portfolio

Objektorientierte und funktionale Programmierung mit Python
DLBDSOOFPP01_D

IU Internationale Hochschule

Studiengang: Cyber Security

Dashboard für das Studium - Konzeptionsphase

Marius Burkhardt

Matrikelnummer: IU14107468

Tutor: Aiko Walte

Abgabedatum: 09.11.2026

Inhalt

Abbildungsverzeichnis III

Einleitung III

1 Ziele des Dashboards 1

2 Dashboard-Skizze 2

2.1 Skizzenbeschreibung 2

3 UML-Klassendiagramm 3

3.1 Beschreibung und Erklärung des Diagramms 3

4 Machbarkeitsprüfung 5

Literaturverzeichnis 6

Abbildungsverzeichnis

Abbildung 1: Tabelle Dashboard-Ziele 1

Abbildung 2: Dashboard-Skizze 2

Abbildung 3: UML-Diagramm der Entity-Klassen 3

Abbildung 4: Tabelle Machbarkeit 5

Einleitung

Dieses Dokument beschreibt die Konzeptionsphase (Phase 1) einer Anwendung zur Verwaltung des Studienfortschritts im Studiengang Cyber Security (B. Sc.). Zentrale Entwurfsentscheidungen sind die Modellierung von vier Entitätsklassen gemäß Vorgabe, SQLite zur persistenten Speicherung sowie Tkinter für die grafische Oberfläche – beides Bestandteile der Python-Standardbibliothek und damit ohne zusätzliche Abhängigkeiten umsetzbar. Das Dashboard-Konzept sieht eine Ampel-basierte Fortschrittsanzeige sowie eine automatische Berechnung von Notendurchschnitt und aktuellem Semester vor.

1 Ziele des Dashboards

Für Studierende der IU soll ein Dashboard entwickelt werden, welches der eigenständigen Verwaltung und Visualisierung des individuellen Fortschritts des Studiengangs dienen soll. Die Grundlage jeder Zielsetzung ist die Struktur des Studiengangs. Jedes Modul ist mit einer Prüfungsleistung verbunden, unabhängig von der jeweiligen Form (Workbook, Klausur, Fallstudie, Präsentation, Portfolio und weitere). Jede Prüfungsleistung wird an der IU mit genau 5 ECTS-Punkten gewertet, bis auf wenige Ausnahmen, z.B. Praktikum und Bachelorarbeit. Auf diese Ausnahmen wird in diesem Portfolio nicht näher eingegangen, da dies die Datenmodellierung erheblich vereinfacht. Folgende Ziele wurden festgelegt:

Angestrebte Zielnote	Angestrebte Zielnote als Durchschnitt über alle bestandenen Prüfungsleistungen. Für die Notenbewertung gilt die an der IU übliche Regel, dass eine Note größer als 4,0 als nicht bestanden zählt; der gültige Wertebereich für eine bestandene Note liegt somit zwischen 1,0 und 4,0. (IU Internationale Hochschule, 2025). Diese Grenzen fließen direkt als Validierungsregel in die Datenmodellierung ein. Automatische Berechnung des Notendurchschnitts auf Basis bestandener Prüfungsleistungen (IU Internationale Hochschule, 2025)
Gesamtfort-schritt ECTS	Gemessen wird der Fortschritt anhand der Gesamtanzahl der zu erreichenden ECTS-Punkten, sowie der bisher erreichten ECTS-Punkte. Hinweis zu ECTS-Punkten: Nach der Prüfungsordnung der IU werden ECTS-Punkte je Modul standardmäßig mit 5 ECTS angesetzt, mit wenigen Ausnahmen wie Praktikum oder Bachelorarbeit (IU Internationale Hochschule, 2025). Dies muss nicht durchgängig für alle Studiengänge gelten, daher bleibt das Attribut *ects* in der Klasse *Modul* bewusst änderbar, mit 5 ECTS lediglich als Standardwert.
Geplante Studiendauer in Semestern	Automatische Ableitung des aktuellen Semesters aus dem Studienstart-Datum (studienstart-Attribut in Studiengang), unabhängig von der separat geforderten Semester-Klasse. Ampelartige Gesamtübersicht über den Studienfortschritt (siehe Abschnitt 2).
Module & Übersicht	Erfassung aller Module inkl. zugehöriger Prüfungsleistung (1:1-Beziehung, siehe Abschnitt 3)

Abbildung 1: Tabelle Dashboard-Ziele

2 Dashboard-Skizze

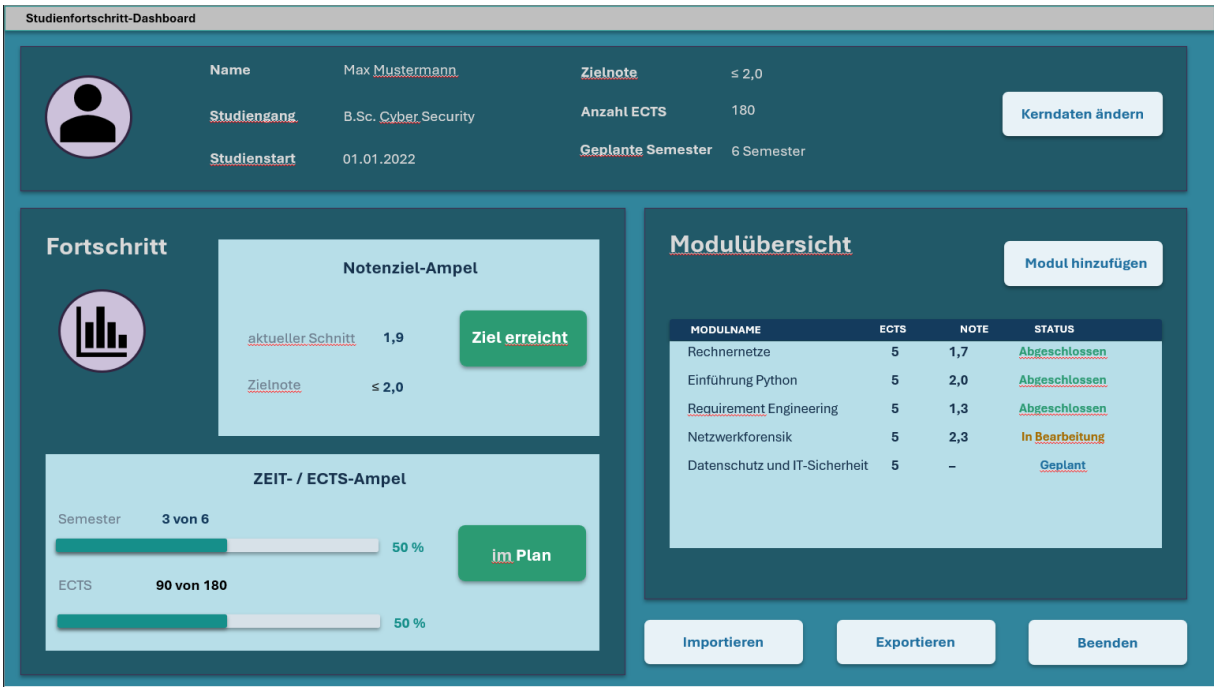


Abbildung 2: Dashboard-Skizze

2.1 Skizzenbeschreibung

Für die Visualisierung des Studienfortschritts wurde eine Ampellogik auf Gesamtebene festgelegt: Der Status des Studiengangs wird nicht je Modul einzeln farblich hervorgehoben, sondern als aggregierte Gesamtansicht dargestellt, die den Fortschritt in Bezug auf die definierten Zielgrößen (verbleibende Semester, erreichte ECTS im Verhältnis zum Soll, aktueller Notendurchschnitt im Verhältnis zur Zielnote) auf einen Blick erfassbar macht. Diese Entscheidung wurde bewusst getroffen, um die Übersichtlichkeit zu wahren; eine Ampel pro einzelnes Modul hätte bei einer wachsenden Anzahl an Modulen schnell zu unübersichtlicher Redundanz geführt, ohne einen zusätzlichen Erkenntnisgewinn zu liefern.

Der Studienfortschritt in Semestern wird nicht manuell gepflegt, sondern automatisiert aus dem Studienstart-Datum berechnet: Anhand der verstrichenen Zeit seit Studienbeginn lässt sich das aktuelle Semester ableiten, ohne dass eine manuelle Aktualisierung durch die Nutzerin oder den Nutzer erforderlich ist. Diese Automatisierung reduziert nicht nur den Pflegeaufwand, sondern macht eine gesonderte Versionierung oder Historisierung des Attributs *aktuelles_semester* obsolet, da der Wert zu jedem Zeitpunkt reproduzierbar aus dem unveränderlichen Startdatum berechnet werden kann.

3 UML-Klassendiagramm

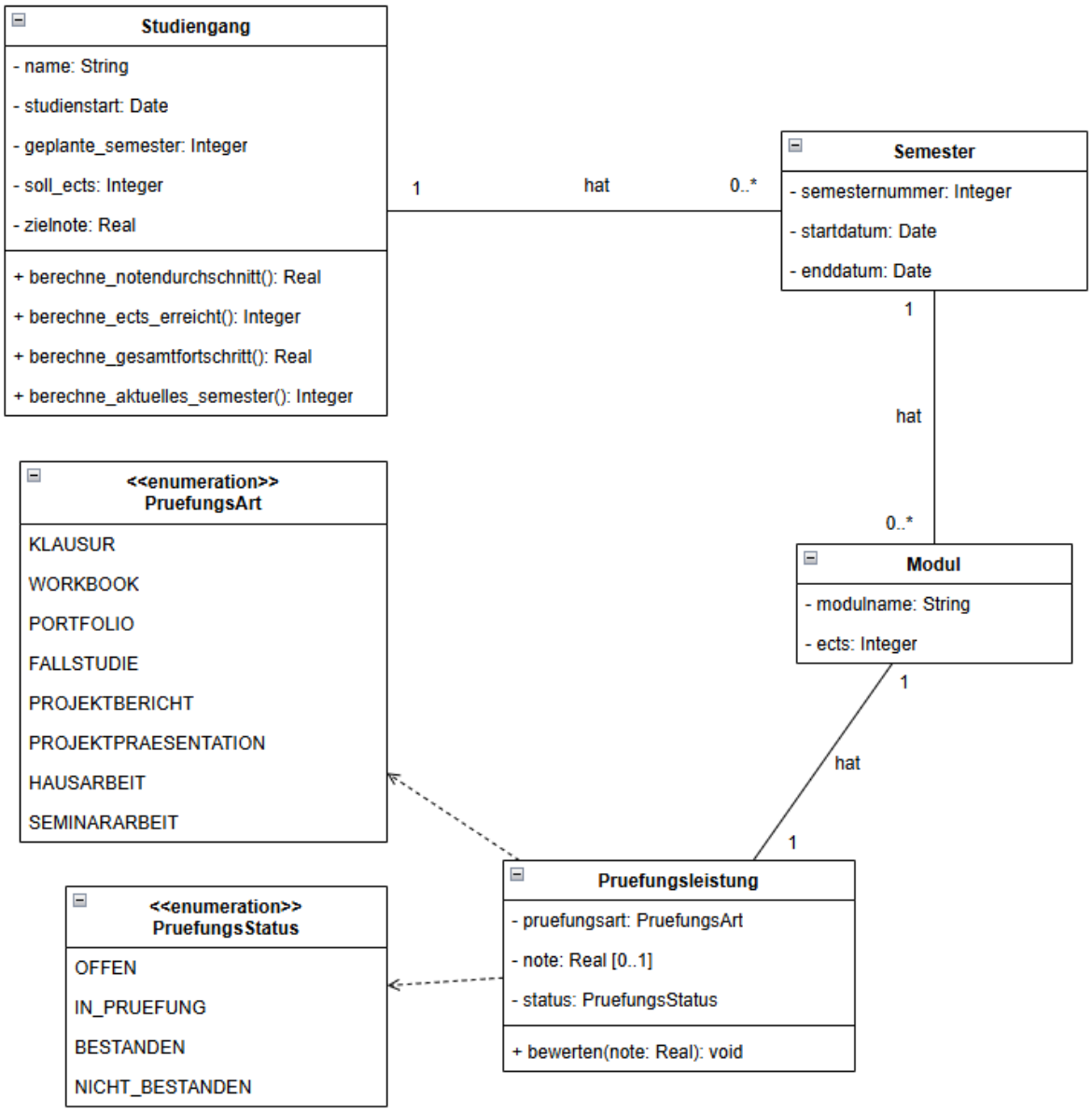


Abbildung 3: UML-Diagramm der Entity-Klassen

3.1 Beschreibung und Erklärung des Diagramms

Klasse Studiengang: Repräsentiert den gesamten Studiengang. Enthält u. a. *name*, *studienstart* (Datum) und weitere Stammdaten. Weiterhin enthält diese Klasse abgeleitete/berechnete Operationen wie Notendurchschnitt, erreichte ECTS, Berechnung zum aktuellen Semester und den Gesamtfortschritt.

Klasse Semester: Enthält die Semester, die zum Studiengang gehören. Ein Semester gehört zu genau einem Studiengang und einem Studiengang können n Semester zugeordnet werden. Darin enthalten sind die Attribute *semesternummer*, *startdatum* und *enddatum*.

Klasse Modul: Bildet ein einzelnes Modul ab und wird der Klasse Semester mit einer 1:0..*-Beziehung zugeordnet. Der Klasse Modul sind die Attribute *modulname* und *ects* (Default 5, änderbar) zugeordnet. Es existiert eine 1:1-Beziehung zu genau einer Prüfungsleistung. Enthält keine eigene Bestehenslogik – diese liegt vollständig in Prüfungsleistung.

Klasse Pruefungsleistung: Enthält *pruefungsart* mit dem Enum *PruefungsArt*. Das Attribut *note* ist optional (Multiplizität 0..1), solange die Prüfungsleistung nicht BESTANDEN ist, da eine Note erst nach einer bestandenen Prüfungsleistung vorliegt (Wertebereich 1,0-4,0). Weiterhin enthält die Klasse das Attribut *status* vom Typ *PruefungsStatus* (Enum). Die 1:1-Beziehung zu Modul stellt sicher, dass jedes Modul genau eine zugehörige Prüfungsleistung besitzt – eine bewusste Modellierungsentscheidung, um Bewertungsdetails nicht in Modul selbst zu vermischen.

PruefungsStatus (Enum): Vier Ausprägungen: OFFEN, IN_PRUEFUNG, BESTANDEN, NICHT_BESTANDEN.

PruefungsArt (Enum): Ausprägung in Summe der verschiedenen Prüfungsleistungen: KLAUSUR, WORKBOOK, PORTFOLIO, FALLSTUDIE, PROJEKTBERICHT, PROJEKTPRAESENTATION, HAUSARBEIT, SEMINARARBEIT.

Hinweis zu Getter/Setter-Methoden: Diese werden in der Klassenbeschreibung nicht einzeln aufgeführt, da sie für nahezu jedes private Attribut vorhanden sind und ihre vollständige Auflistung keinen zusätzlichen Erkenntniswert für die fachliche Struktur liefert.

4 Machbarkeitsprüfung

Kategorie	Auswahl	Begründung
GUI	Tkinter	Tkinter (Python-Standardbibliothek) Keine bzw. kaum externe Bibliotheken notwendig, plattformunabhängig, ausreichend für ein Dashboard mit Tabellen- und Statusanzeige (Python Software Foundation, o. J. b)
Entwicklungs-umgebung	VS Code, ver.1.137.0	Einfache, aber umfangreiche IDE für viele Anwendungsfälle und Bearbeitung des Quellcodes, kostenlos verfügbar
Daten	SQLite	Persistente Datenspeicherung (sqlite3, Python-Standardbibliothek) Bildet die relationalen 1:n- und 1:1-Beziehungen des Datenmodells direkt ab; keine externe Abhängigkeit nötig, da Teil der Standardbibliothek (Python Software Foundation, o. J. a)
Quellcode-verwaltung	GitHub	Versionierung über GitHub - Ermöglicht Nachvollziehbarkeit von Änderungen, Backup und ggf. spätere Zusammenarbeit
Sprache	Python	Vorausgesetzt durch Aufgabenstellung/Modulkontext, gute Standardbibliotheksabdeckung für beide obigen Anforderungen

Abbildung 4: Tabelle Machbarkeit

Fazit (eigene Schlussfolgerung): Da sowohl sqlite3 als auch tkinter Teil der Python-Standardbibliothek sind, ist die Machbarkeit ohne zusätzliche Abhängigkeiten gegeben. Der anfängliche Einfachheitsvorteil einer JSON-Lösung entfällt dadurch weitgehend, während SQLite strukturelle Vorteile für das relationale Datenmodell bietet. Um die Lesbarkeit und Wartbarkeit des Quellcodes zu gewährleisten, folgt die Implementierung den Best Practices des PEP8 Styleguides (van Rossum et al., 2001).

Literaturverzeichnis

IU Internationale Hochschule. (2025). Allgemeine Prüfungsordnung und Modulhandbuch Studiengang Cyber Security (B. Sc.). Gültig ab 19.12.2025

Python Software Foundation. (o. J. a). sqlite3 — DB-API 2.0 interface for SQLite databases. Python 3 Documentation. https://docs.python.org/3/library/sqlite3.html

Python Software Foundation. (o. J. b). *tkinter — Python interface to Tcl/Tk*. Python 3 Documentation. https://docs.python.org/3/library/tkinter.html

van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style Guide for Python Code*. Python Enhancement Proposals. https://peps.python.org/pep-0008/